



Data Handling: Import, Cleaning and Visualisation

Lecture 3: A Brief Introduction to Data and Data Processing

Dr. Aurélien Sallin

2025-09-10

Welcome back!



How was the pace last time?

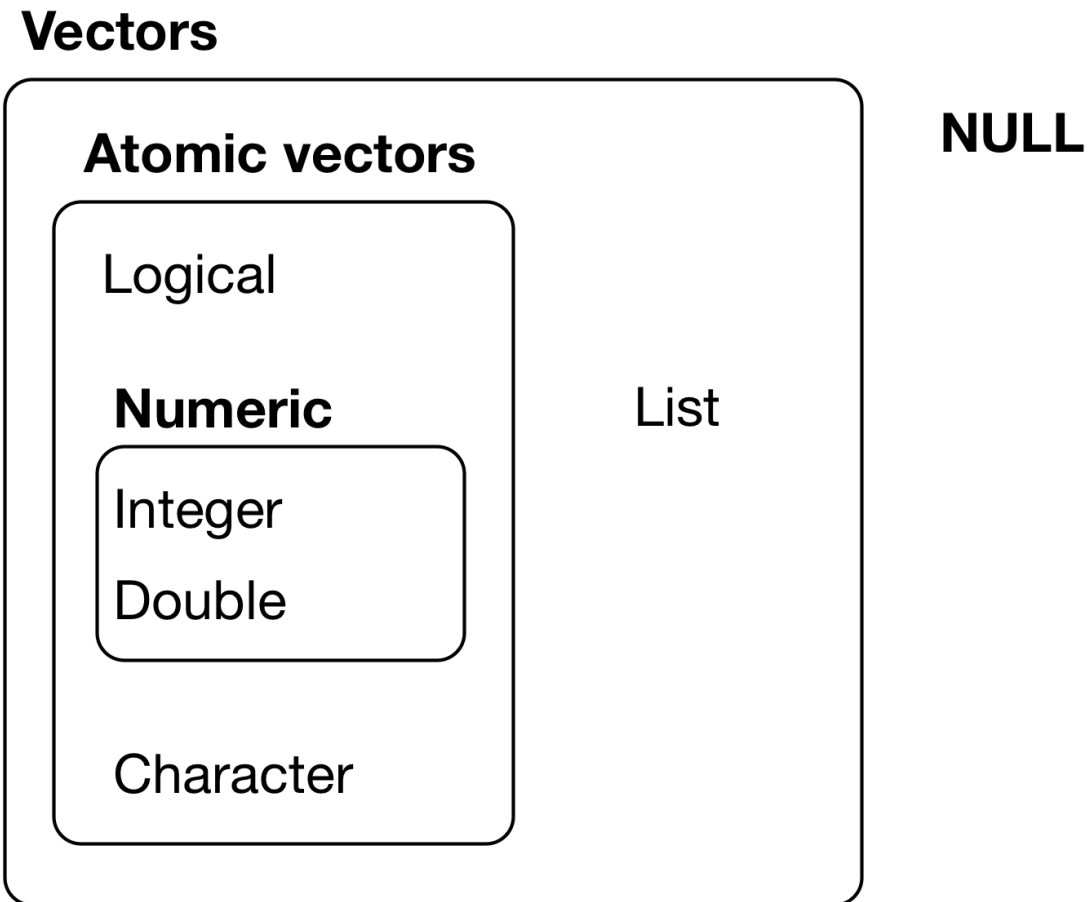
Any comments on what felt too fast or too slow?

Recap

Goals of last course and exercise session

- Basic programming concepts.
- Functions and functionals.
- Working installation of R and first coding exercises.

Atomic Vectors are the most fundamental data structures in R



Source: <https://r4ds.had.co.nz/vectors.html>

Basic programming concepts

- Atomic Vectors
- Operators
- Matrices
- Loops
- Logical statements
- Control statements
- Functions
- Functionals

Three tutorials

- Compute the mean with your own function -> **no solution posted**
- Evolution in action: fast and slow sloths -> **solution in the first exercise session and on Canvas**
- Append and lists -> **solution in the second exercise session and on Canvas**



Information

Examination

Examination for *HSG students in exchange*

For HSG-students currently abroad on an exchange semester (not exchange students at HSG):

- Central examination (50%): as planned, with all course participants.
- Group project (35%): completed remotely with your group.
- Decentral exam (15%): oral examination during the central examination period.

Examination for *exchange students at the HSG*

- Decentral examination (50%): December 18th.
- Group project (35%): completed remotely with your group.
- Decentral exam (15%): December 11th.

Group size for the project

5*

Warm-up

Warm-up

Answer the three questions in this week's quiz.

- Exam style questions.
- Use LLM to help you understand the answer.
- Ask your instructor or TA if you have questions.

Warm-up

Consider the following vector. Which answers are true?

```
some_numbers <- c(30, 50, 60)
```

Possible answers

1. `sum(some_numbers[c(2,3)]) == 80`
2. `some_numbers[some_numbers %in% c(30)] == 30`
3. `some_numbers * 5` will do $30*5 + 50*5 + 60*5$

Warm-up

We create the following matrix

```
mymatrix <- matrix(c(1,2,3,11,12,13,1,10),  
                    nrow = 2,  
                    ncol = 4,  
                    byrow = FALSE)
```

Which of the answers are correct?

Answer 1 (recap lecture 2)

```
max(mymatrix[, 2]) == 3
```

is **TRUE**

Answer 2 (recap lecture 2)

```
apply(mymatrix, MARGIN = 2, min)
```

returns a vector of size 4.

Warm-up

We create the following matrix

```
mymatrix <- matrix(c(1,2,3,11,12,13,1,10),  
  nrow = 2,  
  ncol = 4,  
  byrow = FALSE)
```

Which of the answers are correct?

Answer 3 (🌶️)

```
m <- apply(mymatrix, MARGIN = 2, min)
```

can be coded as

```
i <- 1  
m <- c()  
  
while(i < (ncol(mymatrix) + 1)){  
  m <- append(m, min(mymatrix[,i]))  
  i <- i + 1  
}
```

Warm-up

We create the following matrix

```
mymatrix <- matrix(c(1,2,3,11,12,13,1,10),  
  nrow = 2,  
  ncol = 4,  
  byrow = FALSE)
```

Which of the answers are correct?

Answer 4 (🌶)

In this code:

```
i <- 1  
m <- c()  
  
while(i < (ncol(mymatrix) + 1)){  
  m <- append(m, min(mymatrix[,i]))  
  i <- i + 1  
}
```

we actually don't need to create an empty vector m. We just need to save the output of the loop as an object, i.e.:

```
i <- 1  
  
m <- while(i < (ncol(mymatrix) + 1)){  
  append(m, min(mymatrix[,i]))  
  i <- i + 1  
}
```

Warm-up

What is `total_sum`?

```
numbers <- 1:4
total_sum <- 0
n <- length(numbers)

# start loop
for (i in 1:n) {

  if(i %% 2 == 0){
    total_sum <- total_sum + numbers[i]
  } else {
    total_sum <- total_sum + 2*numbers[i]
  }

}
```

Don't forget...

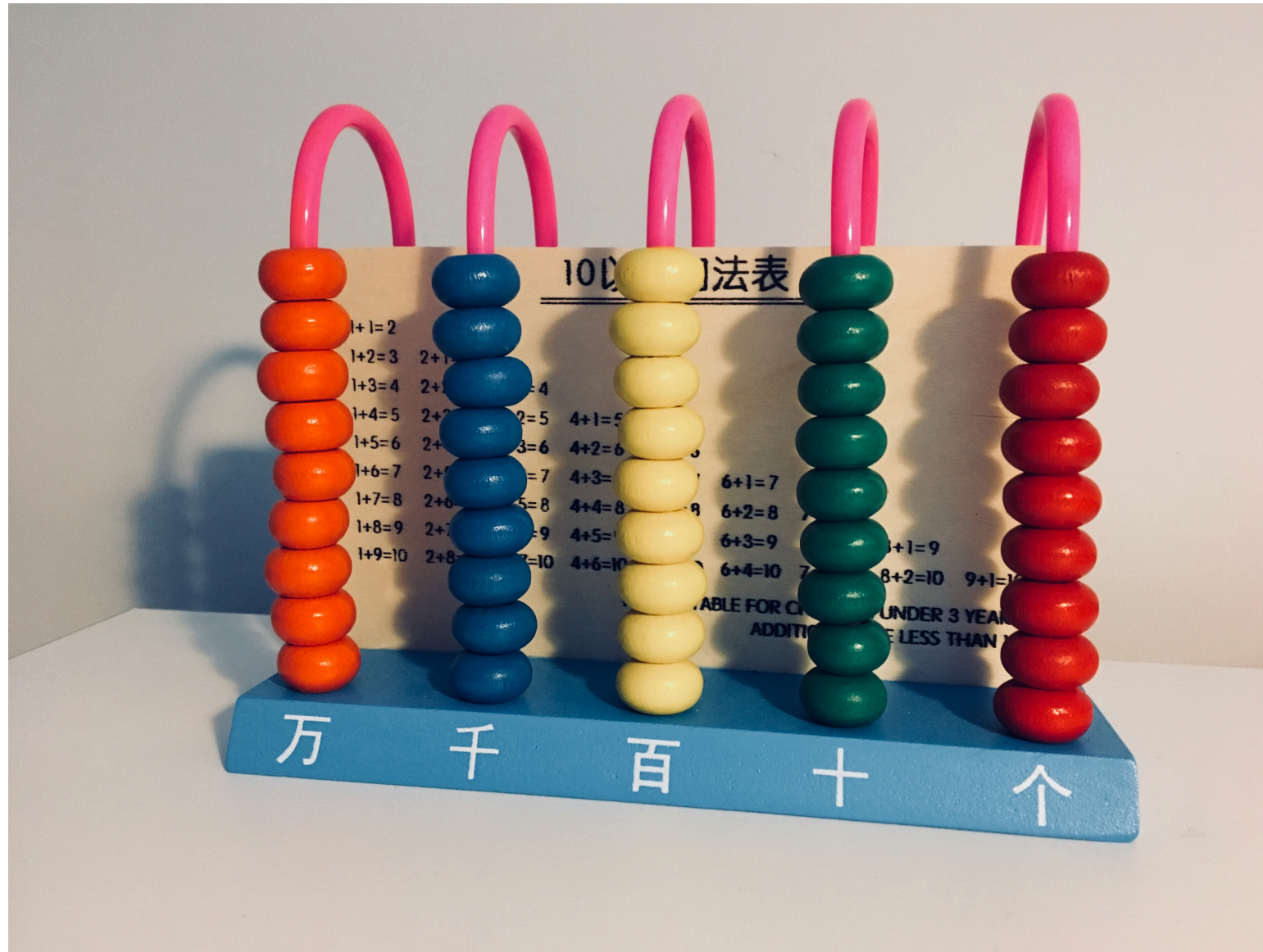


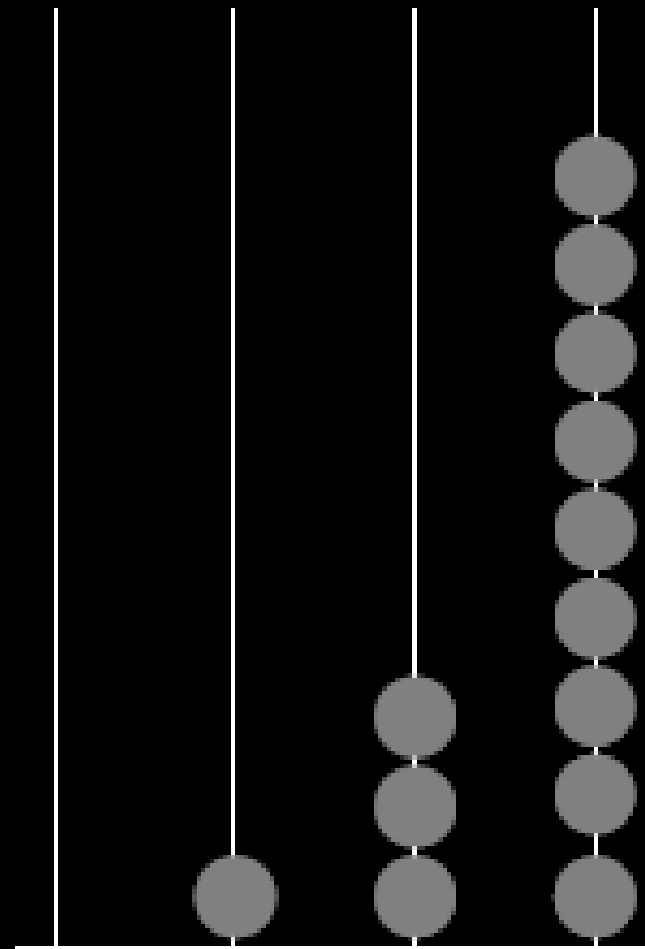
Data Processing

Goals for today

1. Get the basics of **data processing**
2. Learn what **binary** and **hexadecimal** systems are
3. See why **encoding** is crucial for any data project
4. Realize that both code and data are just **text** at their core

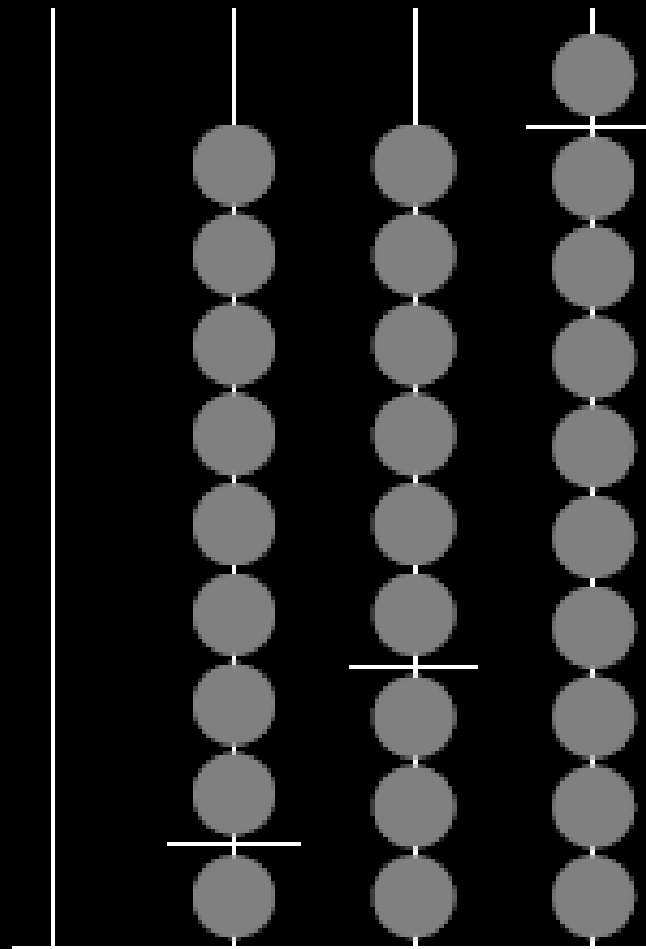
Numerical systems as representations





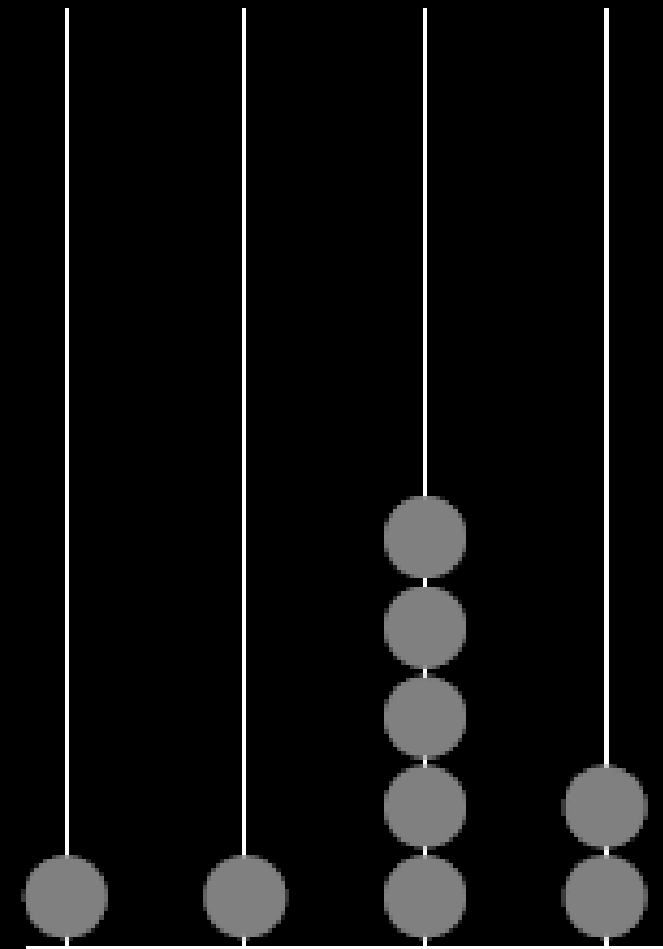
M C X I

139



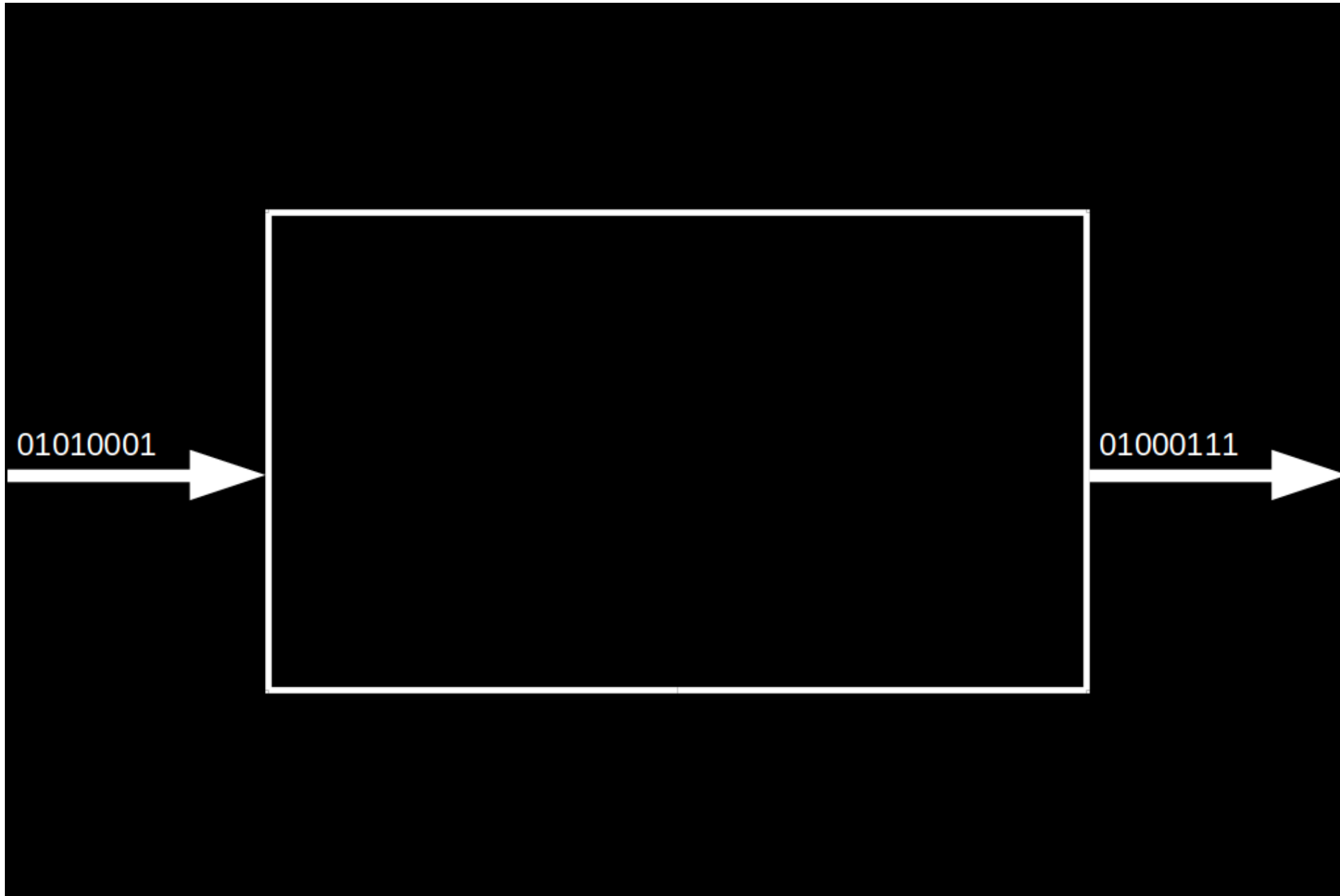
M C X I

139 + 861



M C X I

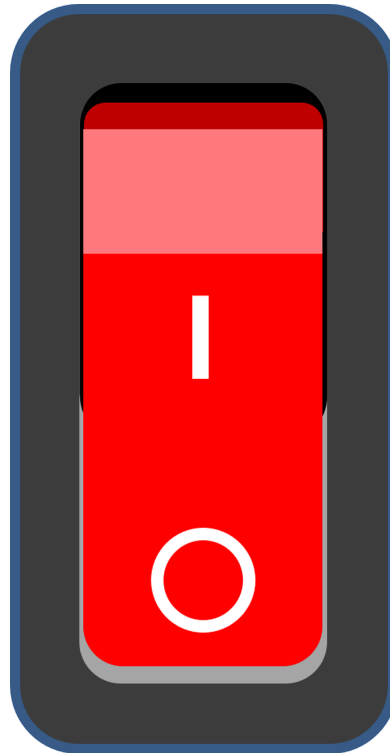
1152



The binary system

Microprocessors can only represent two signs (states):

- 'Off' = 0
- 'On' = 1



The binary system



The binary counting frame

- Only two signs: 0, 1.
- Base 2.
- Columns: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, and so forth.

The binary counting frame

What is the decimal number *139* in the binary counting frame?

The binary counting frame

What is the decimal number *139* in the binary counting frame?

- Solution:

$$(1 \times 2^7) + (1 \times 2^3) + (1 \times 2^1) + (1 \times 2^0) = 139.$$

The binary counting frame

What is the decimal number 139 in the binary counting frame?

- Solution:

$$(1 \times 2^7) + (1 \times 2^3) + (1 \times 2^1) + (1 \times 2^0) = 139.$$

- More precisely:

$$(1 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) \\ + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 139.$$

- That is, the number 139 in the decimal system corresponds to 10001011 in the binary system.

Conversion between binary and decimal

Number	128	64	32	16	8	4	2	1
--------	-----	----	----	----	---	---	---	---

Conversion between binary and decimal

Number	128	64	32	16	8	4	2	1
0 =	0	0	0	0	0	0	0	0
1 =	0	0	0	0	0	0	0	1
2 =	0	0	0	0	0	0	1	0
3 =	0	0	0	0	0	0	1	1
...								
139 =	1	0	0	0	1	0	1	1

The binary counting frame

An algorithm to convert an integer to binary:

1. Divide the integer by 2.
2. Write down the remainder (0 or 1).
3. Update the integer to the quotient (integer division).
4. Repeat until the quotient is 0.
5. The binary number is the sequence of remainders read from bottom to top.

🧠 Challenge: use a **while** loop, the operators **%** and **/%** to implement this algorithm in R.

► Show the function `decimal_to_binary_integers`

```
decimal_to_binary_integers(47)
```

```
[1] "101111"
```

The binary counting frame

- Sufficient to represent all *natural* numbers in the decimal system.
- Representing fractions is tricky
 - e.g. $1/3 = 0.333\dots$ actually constitutes an infinite sequence of 0s and 1s.
 - Solution: 'floating point numbers' (not 100% accurate)

Floating point numbers

```
1 # Subtracting two nearly identical floating-point numbers
2 x <- 0.3 - 0.2
3 y <- 0.1
4
5 # Check if they are equal
6 result <- x == y
```

```
print(x, digits = 20)
```

```
[1] 0.099999999999999977796
```

```
print(y, digits = 20)
```

```
[1] 0.100000000000000000555
```

Floating point numbers

```
1 # Subtracting two nearly identical floating-point numbers
2 x <- 0.3 - 0.2
3 y <- 0.1
4
5 # Check if they are equal
6 result <- x == y
```

```
print(result)
```

```
[1] FALSE
```


Floating point numbers

...now...

```
tolerance <- 1e-9  
equal <- abs(x - y) < tolerance  
print(equal)
```

```
[1] TRUE
```

Floating point numbers

An algorithm to convert a fractional to binary:

1. *Multiply* the fraction by 2.
2. Write down the integer part.
3. Repeat 1 using the fractional part.
4. Repeat until the fractional part is 0.

The binary number is the sequence of remainders read from bottom to top.

Challenge: use a **while** loop, the operators **%** and **/%** to implement this algorithm in R.

► Show the function **decimal_to_binary_fractional**

```
decimal_to_binary_fractional(1/3, max_digits = 20)
```

```
[1] "0.01010101010101010101"
```

Floating point numbers

All in all, we can combine both functions to create a converter function `decimal_to_binary`.

► Show the function `decimal_to_binary`

```
decimal_to_binary(9.25, max_digits = 20)
```

```
[1] "1001.01"
```

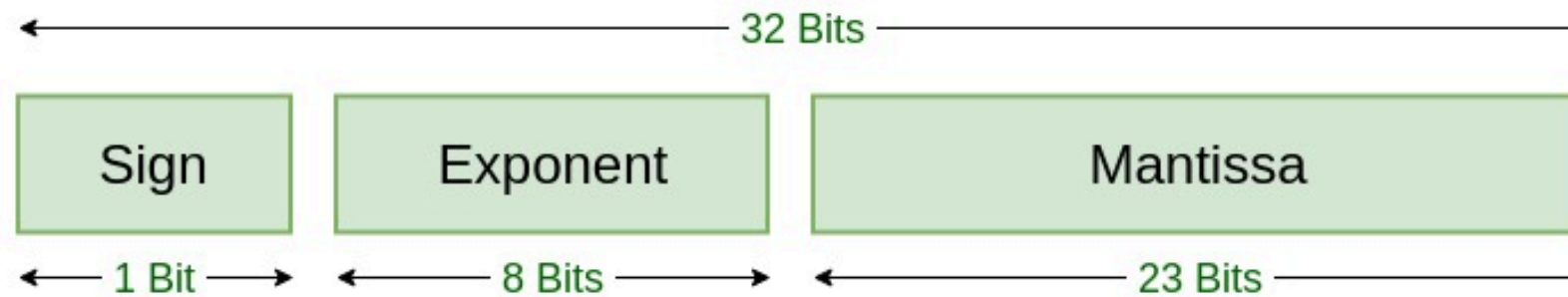
```
decimal_to_binary(1/2, max_digits = 20)
```

```
[1] "0.1"
```

Floating point numbers

? How does the “computer” handle the distinction between the part of a number before and after the decimal point?

- IEEE Standard for Floating-Point Arithmetic (IEEE 754)
- **Not exam relevant.** See [this page](#) for a step-by-step explanation.



Single Precision
IEEE 754 Floating-Point Standard

What time is it?



Floating point numbers be like...

me: computer are awesome, it
can calculate big numbers!

me: *tried programming $0.1 + 0.2$ *

computer:



The hexadecimal system

- Binary numbers can become quite long rather quickly.
- Computer Science: refer to binary numbers with the *hexadecimal* system.

The hexadecimal system

16 symbols >>> base 16

- 0-9 (used like in the decimal system)...
- and A-F (for the numbers 10 to 15).
- each digit represents an increasing power of 16 (16^0 , 16^1 , etc.).

The hexadecimal system

What is the decimal number 139 expressed in the hexadecimal system?

Decimal numbers in hexadecimal

Number	...	$16^2 = 256$	$16^1 = 16$	$16^0 = 1$
0 =	...	0	0	0
1 =	...	0	0	1
15 =	...	0	0	F
16 =	...	0	1	0
256 =	...	1	0	0
...	...			
139 =	0	0	8	B

The hexadecimal system

What is the decimal number 139 expressed in the hexadecimal system?

- Solution:

$$(8 \times 16^1) + (11 \times 16^0) = 139.$$

- More precisely:

$$(8 \times 16^1) + (B \times 16^0) = 8B = 139.$$

- Hence: **10001011** (in binary) = **8B** (in hexadecimal) = **139** in decimal.

The hexadecimal system

Challenge: update your function `decimal_to_binary_integers` to write in hexadecimal system.

► Show the function `decimal_to_binary_integers`

```
lapply(c(1, 2, 12, 48, 255, 256), decimal_to_hexa_integers)
```

```
[[1]]  
[1] "1"  
  
[[2]]  
[1] "2"  
  
[[3]]  
[1] "C"  
  
[[4]]  
[1] "30"  
  
[[5]]  
[1] "FF"  
  
[[6]]  
[1] "100"
```

The hexadecimal system

Advantages (when working with binary numbers)

1. Shorter than raw binary representation
2. Much easier to translate back and forth between binary and hexadecimal than binary and decimal.

WHY?



Character Encoding

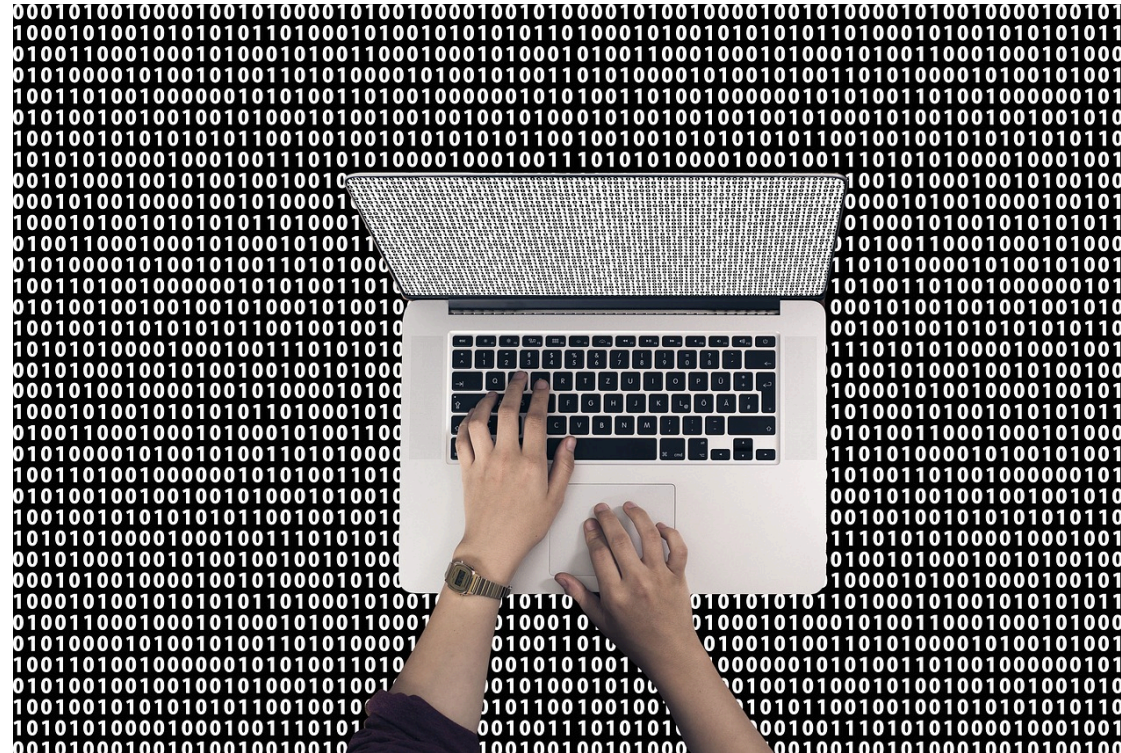
Computers and text

How can a computer understand text if it only understands 0s and 1s?

Computers and text

How can a computer understand text if it only understands 0s and 1s?

- **Standards** define how 0s and 1s correspond to specific letters/characters of different human languages.
- These standards are usually called **character encodings**.



Computers and text

How can a computer understand text if it only understands 0s and 1s?

- **Standards** define how 0s and 1s correspond to specific letters/characters of different human languages.
- These standards are usually called **character encodings**.
- Coded character sets that map unique numbers (in the end in binary coded values) to each character in the set.
- For example, **ASCII** (American Standard Code for Information Interchange), now superseded by **utf-8** (Unicode).



ASCII Table

Binary	Hexadecimal	Decimal	Character
0011 1111	3F	63	?
0100 0001	41	65	A
0110 0010	62	98	b

Character encodings: why should we care?

- In practice, working with data means handling digital data of all formats and shapes.
 - Diverse sources.
 - Different standards.
 - Different languages (Japanese vs English).
 - read and store data.
- At the lowest level, this means understanding/handling encodings.

Computer Code and Text-Files

Putting the pieces together...

Two core themes of this course:

1. How can **data** be **stored** digitally and be **read** by/imported to a computer?
2. How can we give instructions to a computer by writing **computer code**?

Putting the pieces together...

Two core themes of this course:

1. How can **data** be **stored** digitally and be **read** by/imported to a computer?
2. How can we give instructions to a computer by writing **computer code**?

In both of these domains we mainly work with one simple type of document: **text files**.

Text-files

- A **collection of characters** stored in a designated part of the computer memory/hard drive.
- An easy-to-read representation of the underlying information (0s and 1s)!
- Common device to store data:
 - Structured data (tables)
 - Semi-structured data (websites)
 - Unstructured data (plain text)
- Typical device to store computer code.

Unstructured data in text files

- Store “Hello World” in `helloworld.txt`.
 - Allocation of a block of computer memory containing “Hello World!”.
 - Simply a sequence of 0s and 1s...
 - `.txt` indicates to the operating system which program to use when opening this file.
- Encoding and format tell the computer how to interpret the 0s and 1s.

Inspect a text file

Interpreting 0s and 1s as text... using `bash` (scripting language to interact with the operating system):

```
cat helloworld.txt; echo
```

```
Hello World!
```

Inspect a text file

Directly looking at the 0s and 1s...

```
xxd -b helloworld.txt
```

```
00000000: 01001000 01100101 01101100 01101100 01101111 00100000 Hello  
00000006: 01010111 01101111 01110010 01101100 01100100 00100001 World!
```

Inspect a text file

Similarly we can display the content in hexadecimal values:

```
xxd data/helloworld.txt
```

```
00000000: 4865 6c6c 6f20 576f 726c 6421      Hello World!
```

Encoding issues

What is the problem?

```
cat hastamanana.txt; echo
```

```
## Hasta Ma?ana!
```

Encoding issues

```
00000000: 4861 7374 6120 4d61 f161 6e61 210d 0a    Hasta Ma.ana!..
```

Inspect the encoding

```
file -b hastamanana.txt
```

```
ISO-8859 text, with CRLF line terminators
```

f1 is the problematic byte. It corresponds to the “ñ” in ISO-8859. and is the code **U+00F1** in unicode (Latin-1 supplement). In UTF-8, f1 is not a valid character by itself, as UTF-8 expects two bytes (c3 b1).

Read the file again, this time with the correct encoding

```
iconv -f iso-8859-1 -t utf-8 hastamanana.txt | cat
```

```
Hasta Mañana!
```

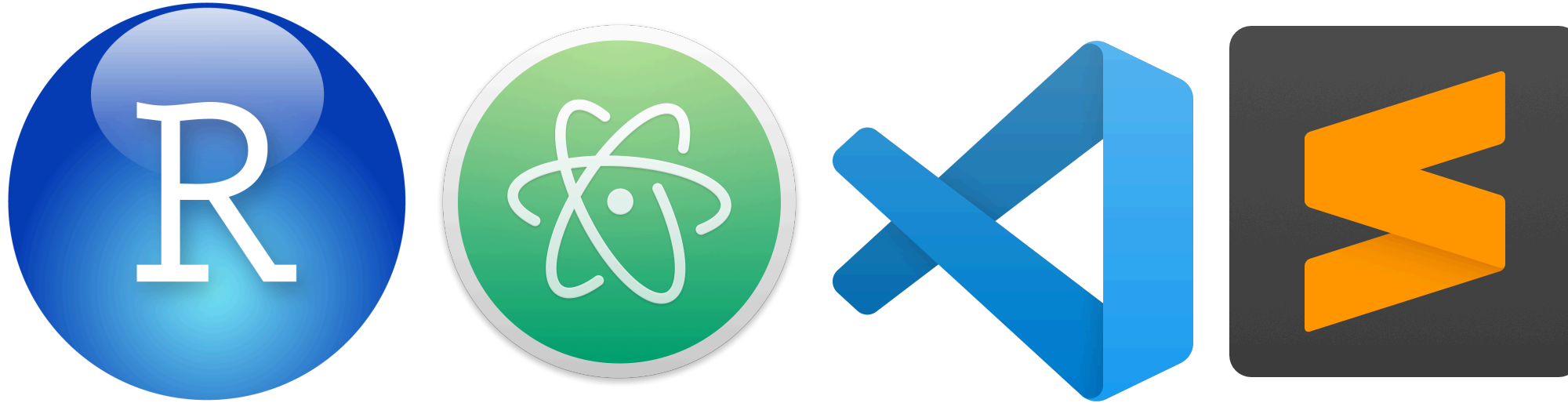
UTF encodings

- 'Universal' standards.
- UTF-8 to UTF-16 to UTF-32
- Contain broad variety of symbols (various languages).
- Less problems with newer data sources...

Take-away message

- Recognize an encoding issue when it occurs!
- Problem occurs right at the beginning of the *data pipeline*!
 - Rest of pipeline affected...
 - ... cleaning of data fails ...
 - ... analysis suffers. 🤖

A side note on text editors: RStudio, Atom, VsCode



Install RStudio from [here](#)!

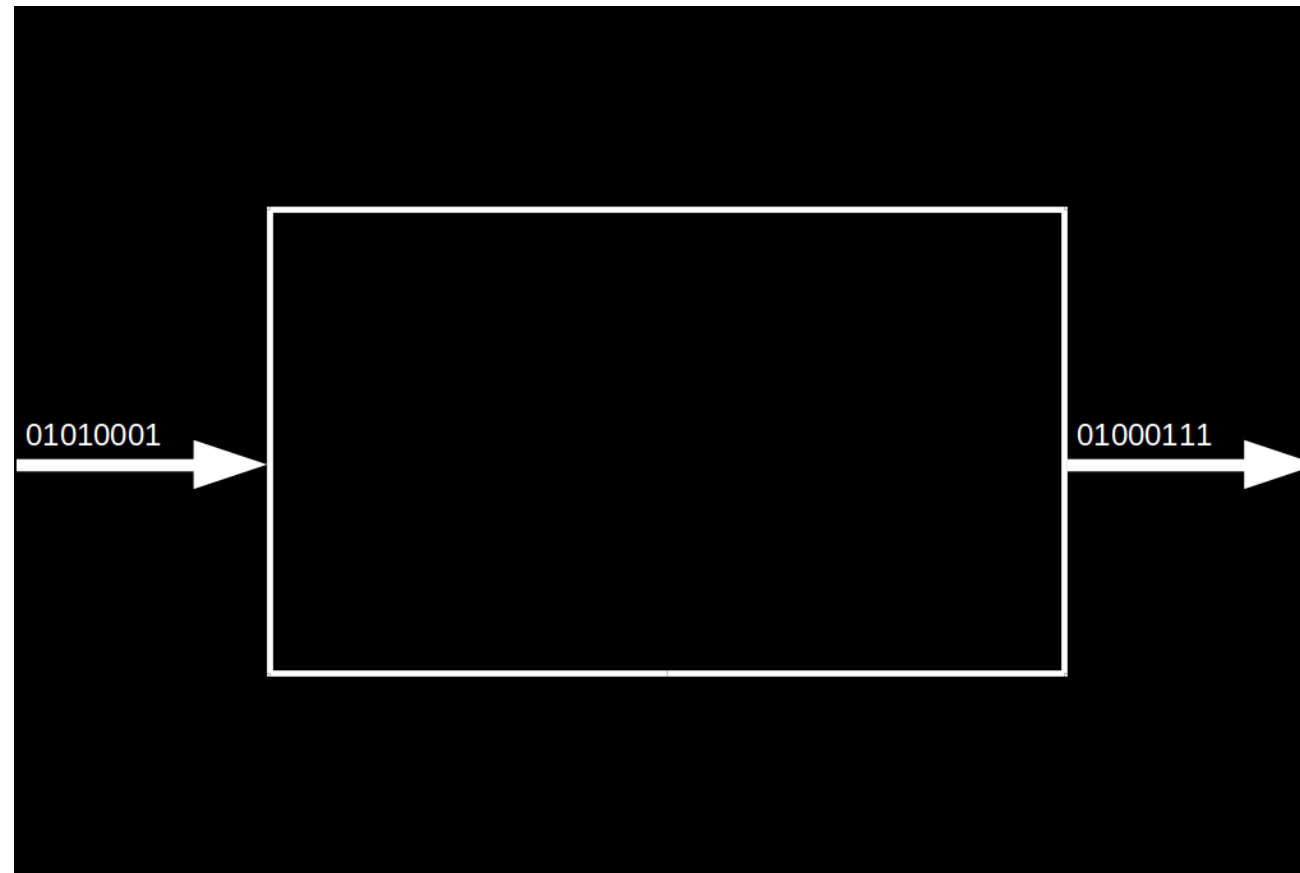
Install Atom from [here](#)!

Install VScode from [here](#)!

Install Sublime text from [here](#)!

Data Processing Basics

Components of a standard computing environment



The 'blackbox' of data processing.

5468616E6B7320616E642073656520796F75206E657874207765656B21



Q&A